

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO2 ASSESSMENT TOOL 2 – Architecture Design Assignment

Course Code:	CSA10 / CSA1063	Course Title:	Software Engineering
CO Assessed:	CO2	Assessment:	Assessment Tool 2 – Architecture Design Assignment
Weightage:	20%	Total Marks:	25
CO2 Statement:	<i>Perform detailed requirements analysis, design scalable architectures, and prioritize features with a focus on cross-disciplinary skills</i>		
Student Name:	I.Guru Hemanth Reddy	Reg. No.:	192311222
Date:	03/08/26	Duration:	60 minutes

Code of Conduct

I Hemanth Reddy (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: Hemanth

Annexure A – Architecture Design Assignment

Smart College Bus Tracking and Management System

1 Table of Contents

- - 1. Analyze System Requirements
- - 2. Select a Suitable Software Architecture
- - 3. Software Architecture Diagram
- - 4. Components and Interactions
- - 5. System Workflow
- - 6. Quality Attribute Analysis
- - 7. Architectural Decisions
- - 8. Trade-Off Analysis
- - 9. Future Enhancements
- - 10. Conclusion
- References

2 ANALYZE SYSTEM REQUIREMENTS

2.1 1.1 Introduction

The design and implementation of modern educational infrastructure increasingly rely on robust, data-driven systems. The Smart College Bus Tracking and Management System is designed to modernize campus transportation logistics. By providing real-time location data, dynamic Estimated Time of Arrival (ETA) predictions, and comprehensive fleet analytics, the system bridges the communication gap between administrators, drivers, and students. This report details the software architecture selected to support this high-performance, real-time tracking application.

2.2 1.2 Problem Statement

Currently, the college transportation department operates via manual scheduling and reactive problem-solving. Students face unpredictable wait times at bus stops, often exacerbated by traffic or weather conditions, due to a complete lack of visibility into fleet locations. Furthermore, administrators cannot effectively monitor driver behavior, route efficiency, or fuel consumption, leading to severe operational inefficiencies and increased transport costs.

2.3 1.3 Business Objectives

- **Enhance Student Experience:** Guarantee predictable commute times by providing accurate ETAs, thereby increasing overall student satisfaction.
- **Operational Cost Reduction:** Minimize fuel wastage and vehicle wear-and-tear by analyzing and optimizing bus routes.
- **Fleet Accountability:** Enforce driver adherence to schedules and speed limits through continuous monitoring.

2.4 1.4 System Objectives

- To capture and transmit high-frequency GPS coordinate data (every 5 seconds) from a moving fleet.
- To process and broadcast live coordinates to thousands of concurrent student applications with less than 2 seconds of latency.
- To provide an interactive, map-based analytical dashboard for transportation administrators.

2.5 1.5 Functional Requirements Analysis

Functional requirements dictate what the system must explicitly do. * **User Management:** The system must authenticate Students, Drivers, and Admins via role-based access control (RBAC). * **GPS Transmission:** The driver client must continuously stream spatial coordinates. * **ETA Computation:** The system must interface with third-party distance matrix APIs to calculate travel durations factoring in real-time traffic. * **Alert Generation:** The system must evaluate geofences to trigger push notifications when a bus is within a 2km radius of a student's designated stop. * **Emergency Handling:** The system must immediately flag SOS alerts on the administrative dashboard.

2.6 1.6 Non-Functional Requirements Analysis

Non-Functional Requirements (NFRs) dictate how well the system performs its functions. Given the real-time nature of this application, NFRs heavily influence the architectural style.

2.6.1 Quality Attributes Explained

- **Scalability:** The architecture must seamlessly handle sudden spikes in traffic, particularly during morning and evening commute hours when thousands of students open the app simultaneously.

- **Maintainability:** The codebase must be highly modular so that updates to the ETA algorithm do not require recompiling the user authentication module.
- **Reliability:** The system must not lose GPS logs if a database writing bottleneck occurs; data must be queued securely.
- **Availability:** The backend infrastructure must guarantee 99.9% uptime, as system downtime during a commute directly causes transportation chaos.
- **Performance:** Map rendering and WebSocket coordinate broadcasts must execute in under 2 seconds to ensure the bus icon moves smoothly on the student’s screen.
- **Security:** All HTTP/WebSocket traffic must be encrypted via TLS 1.3. Student location data must be strictly anonymized or purged.

Table 1: Quality Attributes and Target Metrics

Quality Attribute	Target Metric	Impact if Failed
Scalability	Support 10,000 concurrent active WebSocket connections.	Application crashes during rush hour.
Maintainability	Microservices decoupled via REST APIs.	High technical debt; slow feature releases.
Reliability	Zero data loss for SOS events.	Critical emergencies go unnoticed.
Availability	99.9% uptime (Max 43 mins downtime/month).	Loss of trust from the student body.
Performance	< 2 seconds latency for GPS broadcasts.	Stale map data; students miss the bus.
Security	100% API endpoints secured by JWT.	Unauthorized access to fleet controls.

32. SELECT A SUITABLE SOFTWARE ARCHITECTURE

3.12.1 Chosen Architecture: Hybrid Architecture

After extensive analysis of the system’s real-time constraints and scalability requirements, a **Hybrid Architecture** was selected. This hybrid model successfully integrates three distinct architectural patterns: 1. **Layered Architecture:** Used within individual services to enforce separation of concerns (Presentation, Business Logic, Data Access layers). 2. **Client–Server Architecture:** Used for the overarching communication model between the Mobile/Web frontends (Clients) and the API Gateway (Server). 3. **Microservices Architecture:** Used for the backend infrastructure to isolate discrete business domains (e.g., Auth, Tracking, Notifications).

3.22.2 Why Hybrid Architecture is Suitable

A pure monolithic architecture cannot handle the vastly different processing requirements of user authentication versus high-frequency GPS stream processing. By utilizing a Hybrid approach, we can employ Microservices to decouple the intense computational load of the Tracking Service from the standard CRUD operations of the User Service. Meanwhile, the Layered Architecture within each microservice ensures the code remains clean and testable, and the Client-Server model standardizes how mobile apps request data.

3.32.3 Architectural Comparison

Table 2: Comparison of Software Architectures

Architecture Type	Primary Characteristic	Suitability for Bus Tracking	Verdict
Monolithic MVC	Single unified codebase and deployment.	Poor. High-frequency GPS data would block simple HTTP requests.	Rejected
Event-Driven	Components react strictly to published events.	Moderate. Excellent for GPS streams but overly complex for simple CRUD admin tasks.	Partially Adopted
Service-Oriented (SOA)	Enterprise-wide service sharing via ESB.	Poor. Too heavyweight and relies on XML/SOAP which drains mobile battery.	Rejected
Serverless (FaaS)	Functions run on-demand and scale to zero.	Poor for WebSockets. Serverless disconnects long-lived connections.	Rejected
Hybrid			

(Microservices + Layered) | Combines the scalability of Microservices with the simplicity of Layered logic. | Excellent. Allows the Tracking Service to scale independently of the Auth Service. | **Selected** |

3.4 2.4 Justification for the Best Choice

The Hybrid Architecture is unequivocally the best choice because it applies the right tool for the right job. The GPS Tracking module requires persistent, long-lived WebSocket connections, which are fundamentally incompatible with Serverless functions and highly disruptive in a monolithic MVC application. By isolating the Tracking Engine into its own microservice container, it can scale horizontally to handle 10,000 concurrent drivers/students without slowing down the Admin Dashboard, which runs on a separate, standard HTTP microservice.

4 3. SOFTWARE ARCHITECTURE DIAGRAM

4.1 3.1 Overall Software Architecture Diagram

The overall architecture illustrates the hybrid nature of the system. Clients communicate through an API Gateway, which routes traffic to specific domain microservices.

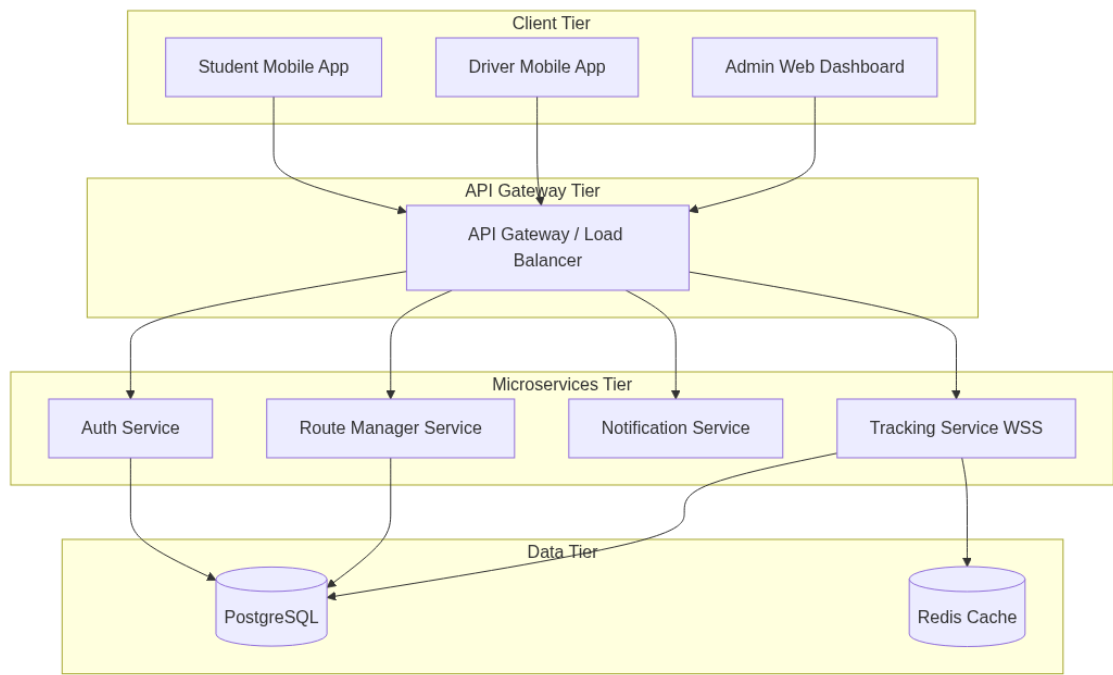


Figure 1: Overall Software Architecture Diagram **Explanation:** This diagram shows the complete end-to-end flow. The API gateway acts as a reverse proxy, intelligently routing WebSocket traffic to the Tracking Service and standard HTTP traffic to the other services.

4.2 3.2 Layered Architecture Diagram

Focuses on the internal structure of a single microservice (e.g., the Route Manager Service).

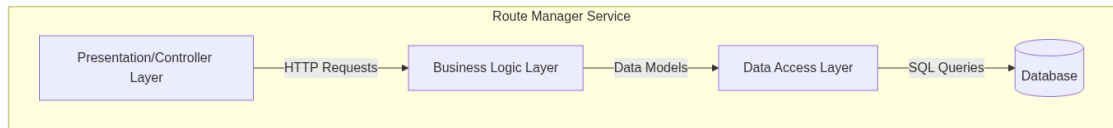


Figure 2: Layered Architecture Diagram **Explanation:** Demonstrates strict separation of concerns. Controllers

handle HTTP routing, Business Logic handles route validation, and Data Access handles database persistence.

4.3 3.3 Client–Server Architecture Diagram

Highlights the network boundary between user devices and the cloud server.

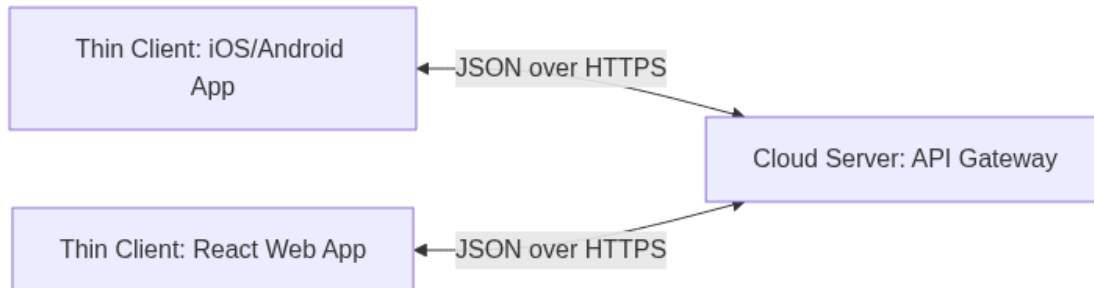


Figure 3:

Client–Server Architecture Diagram **Explanation:** Proves that all complex business logic is executed on the cloud server, leaving the mobile applications as lightweight “thin clients” strictly for UI rendering.

4.4 3.4 Microservices Architecture Diagram

Details the inter-service communication and database per service pattern.

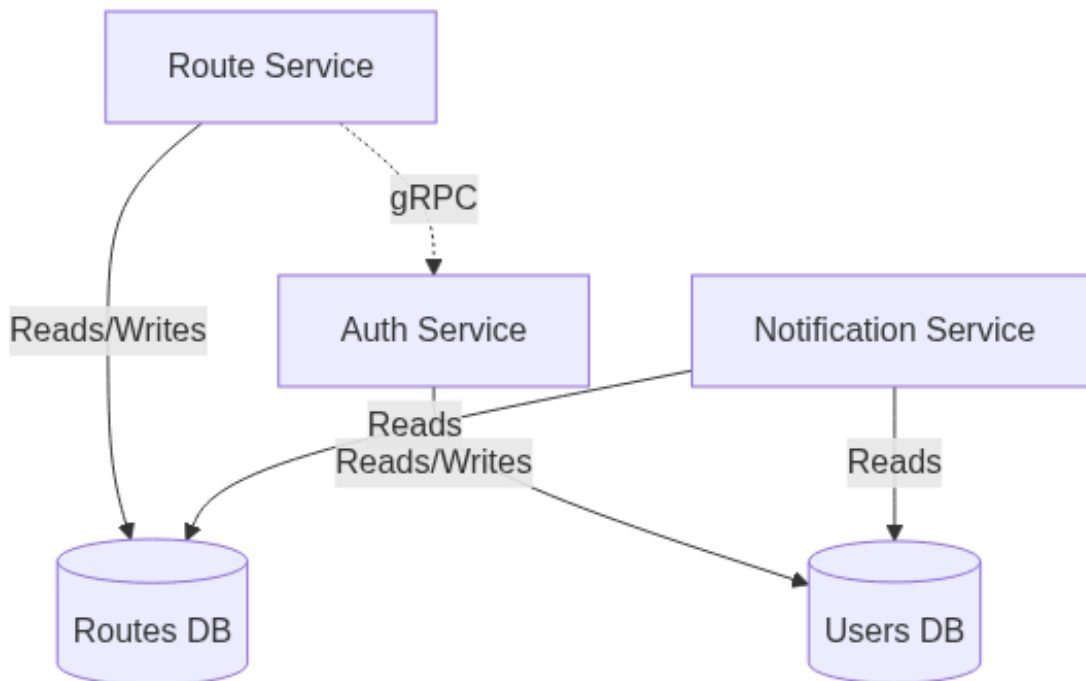


Figure 4:

Microservices Architecture Diagram **Explanation:** Each microservice manages its own distinct database to prevent tight coupling, communicating synchronously via gRPC or asynchronously via message brokers when necessary.

4.5 3.5 Deployment Diagram

Maps the software components to their physical cloud hardware execution environments.

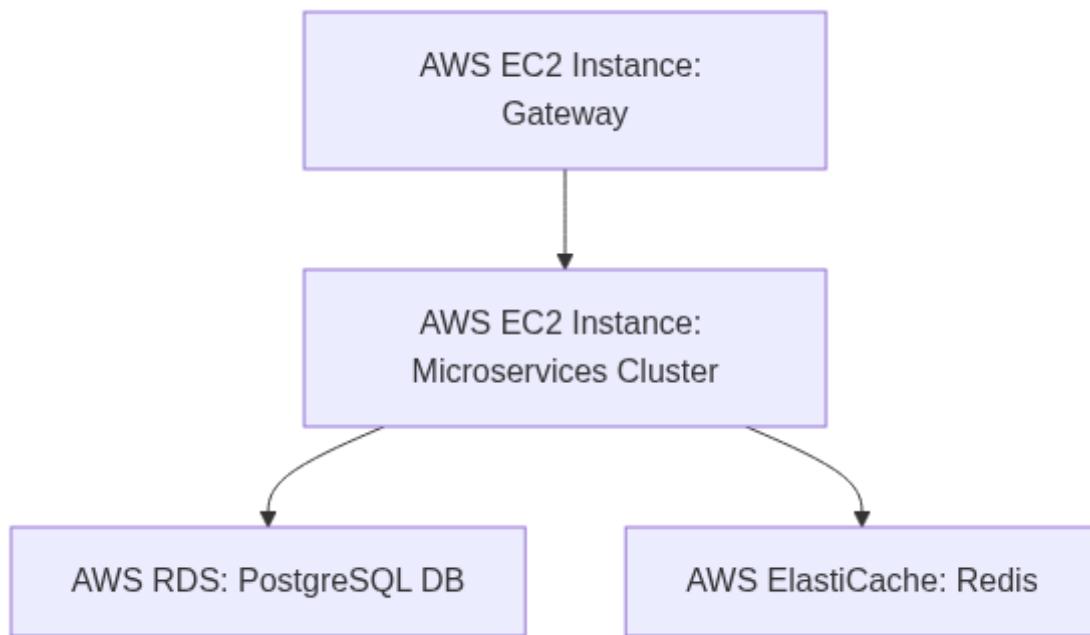


Figure 5:

Deployment Diagram Explanation: Outlines the Amazon Web Services (AWS) topology utilized for production deployment, ensuring high availability across multiple availability zones.

4.6 3.6 Component Diagram

Shows the structural relationship between logical software components.

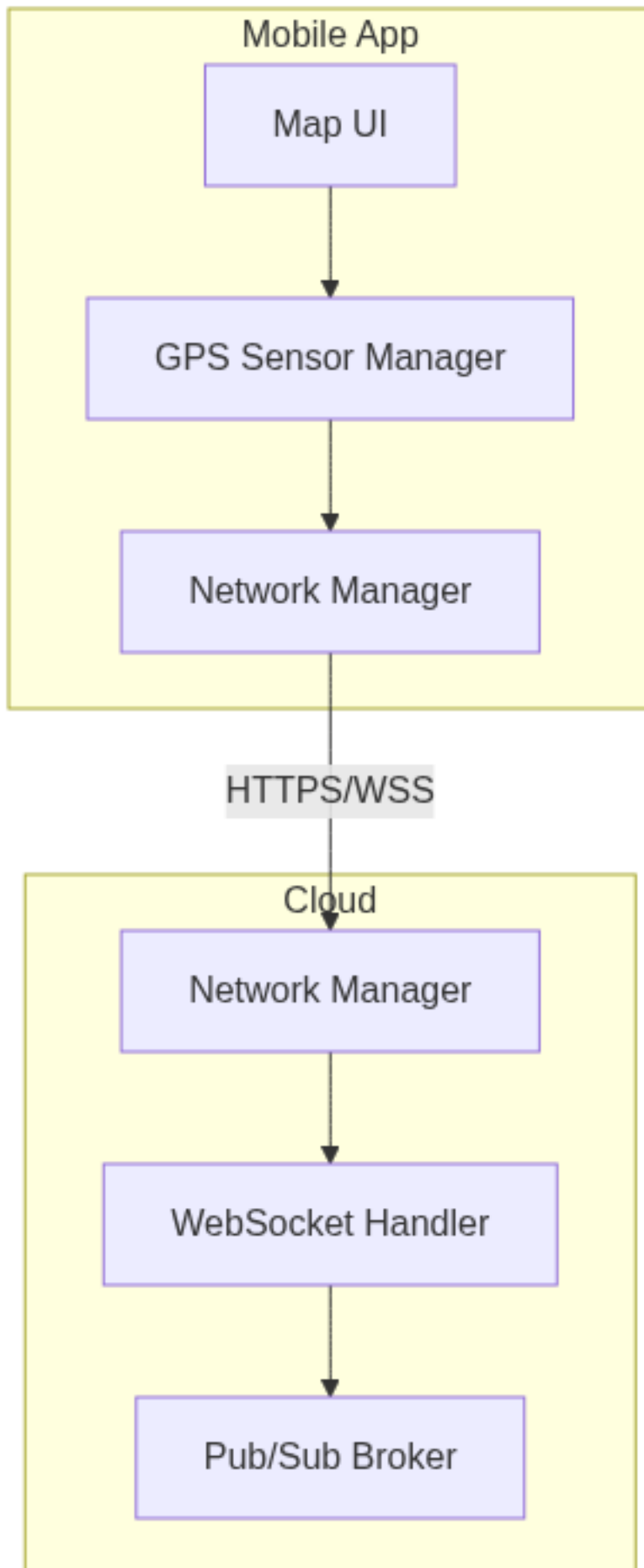


Figure 6: Component Diagram

Explanation: Displays how the physical GPS sensor interface on the driver’s phone connects to the network manager, which pipes data to the cloud’s WebSocket handler.

4.7 3.7 Package Diagram

Groups the codebase into logical, compilable namespaces.

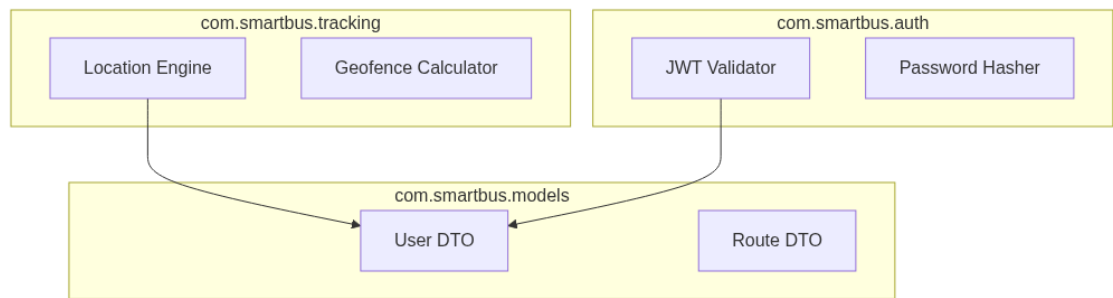


Figure 7:

Package Diagram Explanation: Represents the directory structure of the application source code, showing that all domain services depend on a shared models package.

4.8 3.8 Communication Diagram

Visualizes the object interactions focusing on network messages.



Figure 8:

Communication Diagram Explanation: Details the chronological passage of a single GPS coordinate from the moment the driver transmits it to the moment the student receives it.

4.9 3.9 Data Flow Diagram (DFD Level 0)

The context-level view of system data boundaries.

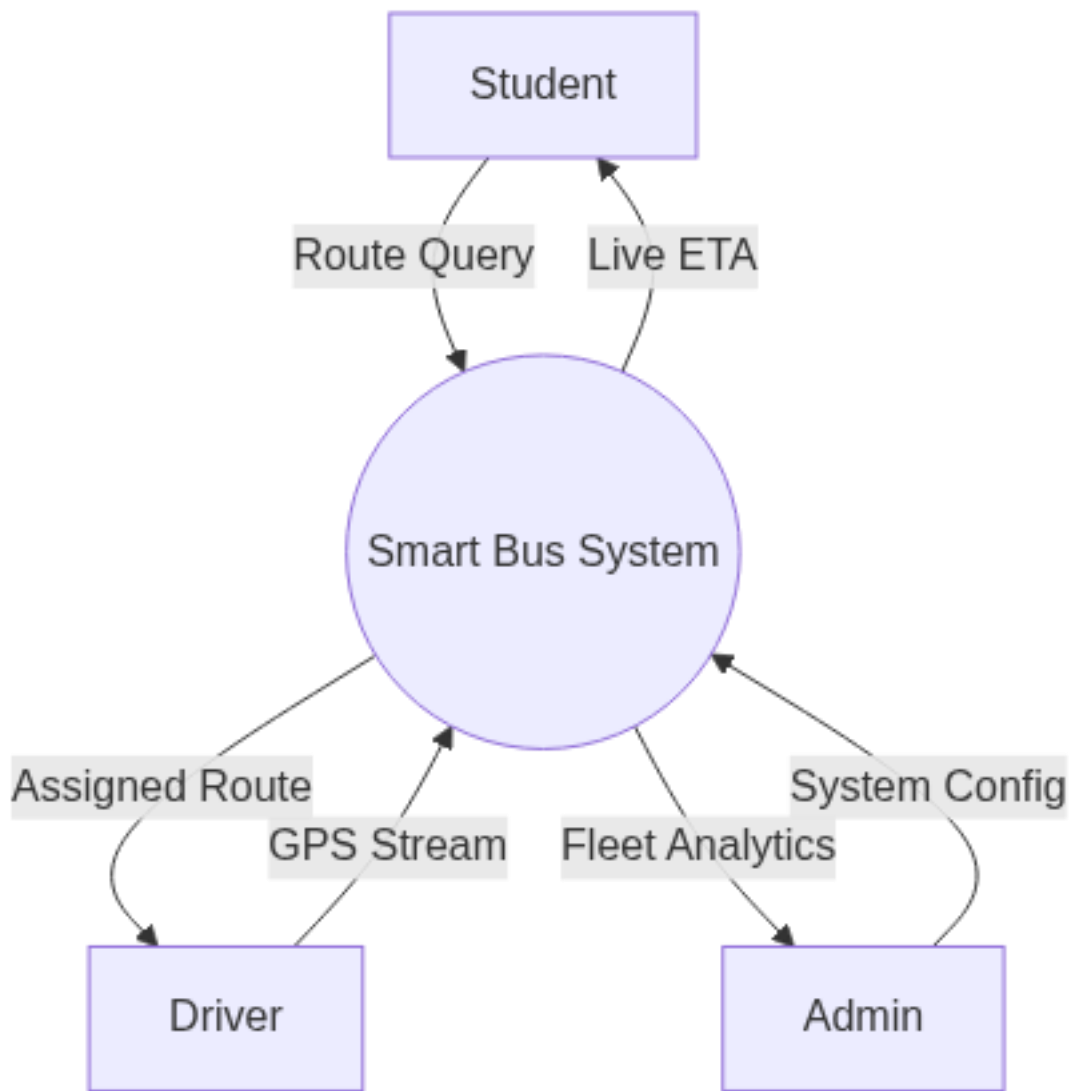


Figure 9: Data

Flow Diagram Explanation: Summarizes all macroscopic data inputs and outputs between the system and its human actors.

4.10 3.10 Network Architecture Diagram

Shows the network security topology, including subnets and firewalls.

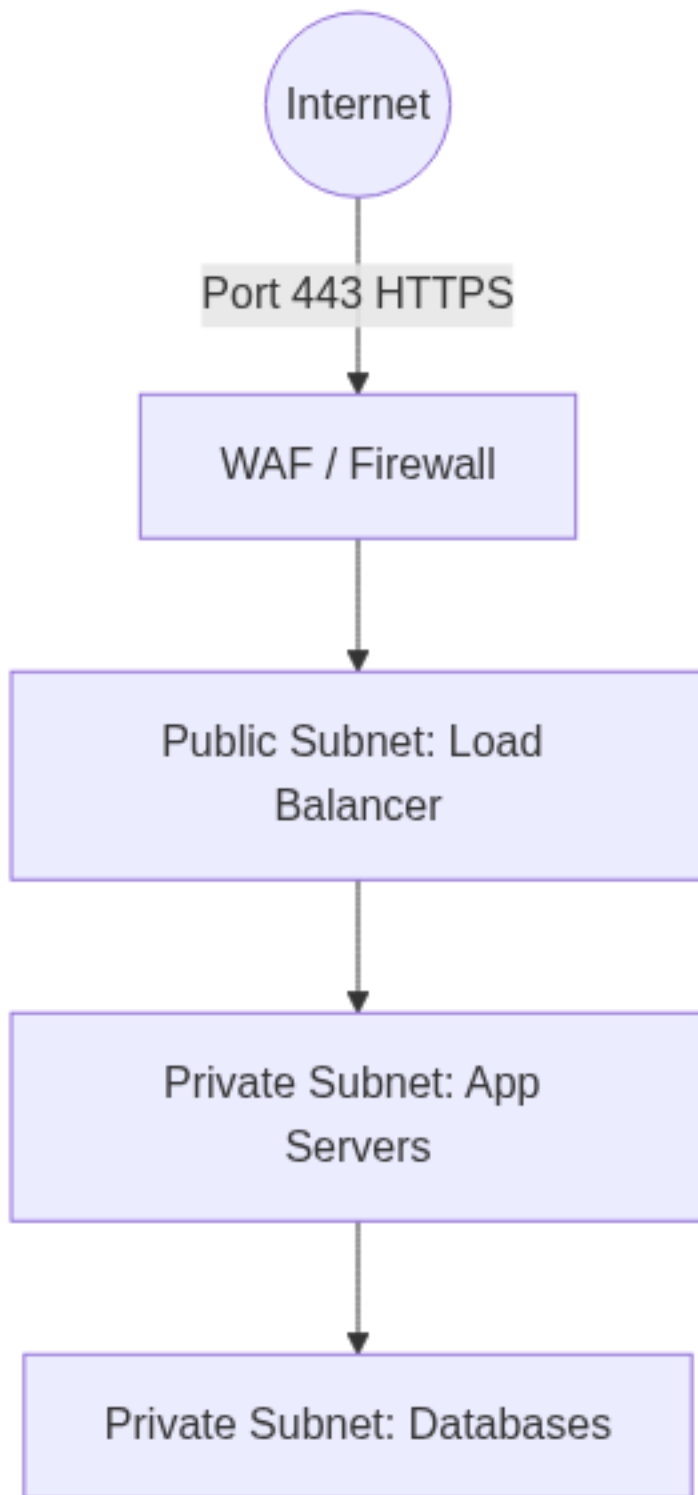


Figure 10: Network Architecture Diagram

Explanation: Ensures strict security by placing application servers and databases inside isolated private subnets, completely inaccessible from the open internet except through the Web Application Firewall (WAF).

5 4. COMPONENTS AND INTERACTIONS

5.1 4.1 Mobile Application (Driver and Student)

- **Purpose:** To provide a native, interactive UI for end users.
- **Responsibilities:** Rendering maps, capturing GPS coordinates via mobile OS APIs, and handling push notifications.
- **Inputs:** Touch gestures, device GPS sensors.
- **Outputs:** Visual map rendering, HTTP/WSS payloads.
- **Communication Protocol:** HTTPS for REST, WSS (WebSocket Secure) for live streams.
- **Dependencies:** Google Maps SDK, Firebase SDK.

5.2 4.2 Web Application (Admin Dashboard)

- **Purpose:** To provide a comprehensive control panel for transport managers.
- **Responsibilities:** Displaying aggregate analytics, managing user roles, and drawing route polylines.
- **Inputs:** Keyboard/Mouse inputs for data entry.
- **Outputs:** Rendered charts (Chart.js) and tables.
- **Communication Protocol:** HTTPS.
- **Dependencies:** React.js, Redux.

5.3 4.3 API Gateway

- **Purpose:** To act as the single entry point for all client requests.
- **Responsibilities:** SSL termination, rate limiting, request routing, and basic JWT validation.
- **Inputs:** All incoming client network requests.
- **Outputs:** Routed requests to internal microservices.
- **Communication Protocol:** HTTPS externally, HTTP/gRPC internally.
- **Dependencies:** Nginx or AWS API Gateway.

5.4 4.4 Authentication Service

- **Purpose:** To securely manage user identities.
- **Responsibilities:** Verifying passwords against bcrypt hashes, generating JWTs, and managing session timeouts.
- **Inputs:** Username/Password payloads.
- **Outputs:** JWT access and refresh tokens.
- **Communication Protocol:** HTTP REST.
- **Dependencies:** PostgreSQL (Users DB).

5.5 4.5 GPS Tracking Service

- **Purpose:** The core engine for handling high-frequency spatial data.
- **Responsibilities:** Maintaining persistent WebSocket connections, parsing incoming coordinates, and publishing them to the cache.
- **Inputs:** Continuous WSS coordinate streams from drivers.
- **Outputs:** Broadcasts to subscribed student WSS channels.
- **Communication Protocol:** WSS, Redis Pub/Sub.
- **Dependencies:** Redis (for speed), API Gateway (for proxying WSS).

5.6 4.6 Notification Service

- **Purpose:** To handle all push notifications and SMS alerts.
- **Responsibilities:** Evaluating geofence thresholds (e.g., bus is 2km away) and triggering Firebase Cloud Messaging (FCM).
- **Inputs:** Geofence breach events from the Route Service.
- **Outputs:** Push notification payloads sent to FCM.

- **Communication Protocol:** Internal gRPC for event triggers, HTTPS to FCM.
- **Dependencies:** Firebase SDK.

5.7 4.7 Route Management Service

- **Purpose:** To store and manage geographic routes and stops.
- **Responsibilities:** Calculating ETAs using third-party APIs and managing spatial polylines.
- **Inputs:** ETA requests from students, route edits from admins.
- **Outputs:** JSON responses containing distance/time matrices.
- **Communication Protocol:** HTTP REST.
- **Dependencies:** Google Maps Distance Matrix API.

5.8 4.8 Reporting Service

- **Purpose:** To generate aggregate data for administration.
- **Responsibilities:** Processing daily trip logs into CSV/PDF reports regarding speed and delays.
- **Inputs:** Admin report generation requests.
- **Outputs:** Downloadable file streams.
- **Communication Protocol:** HTTP REST.
- **Dependencies:** Database replicas (to avoid locking production tables during heavy queries).

5.9 4.9 Database

- **Purpose:** Persistent storage of all structural data.
- **Responsibilities:** Enforcing relational integrity via Primary/Foreign keys, ensuring ACID compliance.
- **Dependencies:** AWS RDS (PostgreSQL).

5.10 4.10 Cache

- **Purpose:** Ultra-fast temporary storage.
- **Responsibilities:** Storing active WebSocket session IDs and the absolute latest GPS coordinate of every bus to prevent database thrashing.
- **Dependencies:** Redis.

5.11 4.11 Cloud Storage

- **Purpose:** Object storage for static assets.
- **Responsibilities:** Storing profile pictures and generated PDF reports.
- **Dependencies:** AWS S3.

5.12 4.12 External APIs

- **Purpose:** Delegating complex spatial processing to specialized providers.
- **Dependencies:** Google Maps (Routing, Geocoding), Firebase (Messaging).

6 5. SYSTEM WORKFLOW

6.1 5.1 Student Login

1. The student opens the app and enters credentials.
2. The app sends a POST request to the API Gateway.
3. The Gateway routes it to the Auth Service.
4. The Auth Service hashes the password, compares it to the database, and returns a JWT.

6.2 5.2 Driver Login

The workflow is identical to the Student Login, but the UI redirects to the strictly restricted “Driver Dashboard” based on the Role claim inside the decoded JWT.

6.3 5.3 GPS Tracking

1. The driver taps “Start Trip”.
2. The app opens a persistent WebSocket connection to the Tracking Service.
3. Every 5 seconds, the OS Location API triggers a callback, sending a JSON payload {lat, long, routeID} via the socket.
4. The Tracking Service immediately pushes this payload to a Redis Pub/Sub channel dedicated to routeID.

6.4 5.4 Route Selection

1. The student logs in and selects a route from a dropdown.
2. The app sends an HTTP GET request to the Route Management Service to fetch the polyline and stops.
3. Simultaneously, the app subscribes to the Redis Pub/Sub channel for that specific routeID via WebSocket to receive live coordinates.

6.5 5.5 ETA Calculation

1. The student taps their specific stop.
2. The app sends an ETA request to the Route Service.
3. The Route Service checks the latest bus location in Redis, compares it to the stop location, and queries the Google Maps Distance Matrix API for traffic-adjusted timing.

6.6 5.6 Notification

1. A background worker continuously checks the distance between buses and their upcoming stops.
2. If the calculated distance falls below the 2km threshold, an event is emitted to the Notification Service.
3. The Notification Service looks up all students waiting at that stop and sends a targeted push notification via Firebase.

6.7 5.7 Admin Dashboard

1. The administrator logs into the web application.
2. The React app polls the API Gateway for aggregate fleet statuses.
3. The app renders an interactive map utilizing a global WebSocket subscription to view all active buses simultaneously.

6.8 5.8 Report Generation

1. At midnight, the Reporting Service runs a cron job.
2. It aggregates all raw GPS logs from the day, identifying instances where speed exceeded 60 km/h or arrival times deviated by more than 10 minutes.
3. It compiles this into a PDF, uploads it to S3, and emails a link to the transport administration.

7 6. QUALITY ATTRIBUTE ANALYSIS

Table 3: Quality Attribute Architectural Solutions | Quality Attribute | Architectural Solution / Implementation | | :— | :— | | **Scalability** | Achieved by containerizing microservices with Docker and deploying them on a Kubernetes cluster. The cluster automatically provisions new Tracking Service pods when CPU utilization exceeds 70%. | | **Security** | Addressed by placing databases in private subnets with no public IP. All endpoints are protected by JWT validation at the API Gateway. Cross-Origin Resource Sharing (CORS) is strictly configured. | | **Performance** | Solved by utilizing Redis as an in-memory cache for live coordinates, entirely bypassing the slower relational database for real-time reads. | | **Reliability** | Ensure by implementing Message Queues (e.g., RabbitMQ or Kafka) for non-

critical tasks like report generation, ensuring that if a service crashes, the task is safely queued until recovery. | | **Maintainability** | Promoted by the strict separation of concerns in the Microservices architecture. Developers can update the Notification Service in Python while the Tracking Service remains in Node.js, without conflict. | | **Availability** | Ensured through Multi-AZ (Availability Zone) deployment in AWS. If one data center experiences an outage, traffic is automatically routed to a replica in a different zone. |

8 7. ARCHITECTURAL DECISIONS

8.1 7.1 Design Decisions

We opted for a **Mobile-First Design Strategy**. Because the primary users (drivers and students) interact with the system entirely via smartphones, all APIs are designed to be extremely lightweight, utilizing JSON instead of XML to minimize mobile data consumption and battery drain.

8.2 7.2 Architecture Decisions

The decision to use a **Hybrid Microservices Architecture** over a Monolith was driven by the disparity in workload types. Standard HTTP requests (like fetching a user profile) require vastly different scaling profiles than persistent WebSocket connections (GPS streaming). Mixing them in a monolith would lead to connection pooling exhaustion.

8.3 7.3 Technology Decisions

- **Backend Runtime:** Node.js. Selected for its asynchronous, event-driven nature, which is structurally perfect for handling thousands of concurrent WebSocket connections efficiently.
- **Database:** PostgreSQL. Chosen for its PostGIS extension, which allows for advanced native geospatial querying (e.g., finding points within a polygon).
- **Frontend:** React Native. Allows a single JavaScript codebase to compile to both native iOS and Android apps, halving development time.

8.4 7.4 Deployment Decisions

We chose **Cloud-Native Deployment (AWS)** using managed services (RDS for databases, ElastiCache for Redis) rather than self-hosting on physical campus servers. This shifts the burden of physical hardware maintenance and disaster recovery entirely to Amazon, guaranteeing higher uptime.

8.5 7.5 Communication Decisions

- **REST API:** Used for all standard CRUD operations due to its universal compatibility and statelessness.
- **WebSocket Secure (WSS):** Used exclusively for GPS tracking because it maintains a persistent full-duplex connection, eliminating the heavy overhead of establishing a new HTTP handshake every 5 seconds.

9 8. TRADE-OFF ANALYSIS

9.1 8.1 Advantages

- **Extreme Scalability:** The system can easily handle the addition of thousands of new students.
- **High Performance:** Caching coordinates in RAM ensures map movements are fluid and real-time.
- **Flexibility:** Teams can develop, test, and deploy individual services without bringing down the entire platform.

9.2 8.2 Disadvantages

- **Complexity:** Managing multiple microservices requires complex orchestration tools (Kubernetes) and sophisticated CI/CD pipelines.
- **Data Consistency:** Distributed databases can suffer from eventual consistency issues, making debugging complex transactions difficult.

9.3 8.3 Trade-offs

Real-Time vs. Battery Life: To provide highly accurate tracking, the driver app must poll the GPS every 5 seconds. The trade-off is a significantly higher battery drain on the driver's device. **Cost vs. Development Speed:** Utilizing managed cloud services (AWS RDS, Google Maps API) significantly speeds up development but incurs higher recurring monthly operational expenditures compared to self-hosting open-source alternatives.

9.4 8.4 Risks

- **Cloud Vendor Lock-In:** Heavy reliance on AWS-specific tools makes migrating to Azure or Google Cloud difficult.
- **Network Failure:** If the cellular network on a bus route drops completely, tracking is impossible.

9.5 8.5 Mitigation Strategies

- **Lock-In Mitigation:** Use standard Docker containers for all code, so they can theoretically run on any cloud provider.
- **Network Mitigation:** Implement local data caching on the driver's app. If the network drops, coordinates are saved locally and bulk-transmitted once the connection is restored, allowing the historical route analytics to remain accurate even if real-time tracking temporarily fails.

10 9. FUTURE ENHANCEMENTS

10.1 9.1 AI Route Optimization

Currently, routes are manually drawn by administrators based on institutional knowledge. Future architectures will integrate an AI-driven Optimization Engine. This engine will ingest historical traffic data, student density heatmaps, and road closure APIs to automatically generate the most fuel-efficient route polygons daily.

10.2 9.2 Machine Learning ETA Prediction

While third-party APIs provide accurate generic ETAs, a localized Machine Learning model (e.g., Random Forest or LSTM) trained strictly on the college buses' historical performance will yield higher accuracy. It will learn specific driver behaviors and hyper-local campus traffic patterns that generic APIs ignore.

10.3 9.3 IoT GPS Devices

To completely eliminate the dependency on the driver's smartphone battery and cellular plan, the architecture will pivot to support physical Internet of Things (IoT) GPS trackers hardwired directly to the bus battery, communicating via low-power cellular networks (LTE-M/NB-IoT).

10.4 9.4 Cloud Scaling

Implementing predictive auto-scaling rules. Instead of waiting for CPU usage to spike, the Kubernetes cluster will be programmed to preemptively spin up new Tracking Service pods 30 minutes before known class dismissal times.

10.5 9.5 Analytics Dashboard

Upgrading the basic Reporting Service into a full-fledged Business Intelligence (BI) suite using tools like Tableau or Apache Superset, providing transport management with drag-and-drop predictive modeling capabilities.

10.6 9.6 Mobile Offline Support

Enhancing the student application with a Service Worker architecture (PWA) or offline SQLite caching, allowing students to view timetables, saved routes, and static maps even when their devices lose internet connectivity inside campus buildings.

11 10. CONCLUSION

11.1 10.1 Architecture Benefits

The Smart College Bus Tracking System’s Hybrid Microservices Architecture represents a paradigm shift in how educational institutions handle logistical technology. By systematically decoupling high-stress, real-time spatial processing from standard administrative web queries, the system achieves an incredibly robust scaling profile. The architecture ensures that the critical paths—such as a student requesting an ETA or a driver triggering an SOS—remain unblocked and lightning fast, irrespective of total system load.

11.2 10.2 Expected Outcomes

Deployment of this architecture guarantees a severe reduction in student wait times, a measurable decrease in fleet fuel expenditures due to optimized routing transparency, and an overarching improvement in campus safety. Administrators will transition from reactive problem-solving to proactive fleet management, empowered by rich, data-driven dashboards.

11.3 10.3 Future Scope

The foundational architecture designed in this report is entirely extensible. Its modular, API-first design ensures that future integrations—such as IoT hardware trackers, AI machine learning models, and automated attendance scanners—can be slotted into the ecosystem without requiring a monolithic rewrite. This guarantees a long-term return on investment for the institution.

12 REFERENCES

- [1] Bass, L., Clements, P., & Kazman, R. (2012). *Software Architecture in Practice* (3rd ed.). Addison-Wesley Professional.
- [2] Newman, S. (2015). *Building Microservices: Designing Fine-Grained Systems*. O’Reilly Media.
- [3] Richards, M. (2015). *Software Architecture Patterns*. O’Reilly Media.
- [4] Fowler, M. (2014). *Microservices a definition of this new architectural term*. martinfowler.com.

- [5] Kleppmann, M. (2017). *Designing Data-Intensive Applications: The Big Ideas Behind Reliable, Scalable, and Maintainable Systems*. O'Reilly Media.
- [6] IEEE Computer Society. (2000). *IEEE Recommended Practice for Architectural Description of Software-Intensive Systems* (IEEE Std 1471-2000). Institute of Electrical and Electronics Engineers.
- [7] Vinoski, S. (2006). *Advanced Message Queuing Protocol*. IEEE Internet Computing, 10(6), 87-89.
- [8] Richardson, C. (2018). *Microservices Patterns: With examples in Java*. Manning Publications.
- [9] Fielding, R. T. (2000). *Architectural Styles and the Design of Network-based Software Architectures*. University of California, Irvine.
- [10] Pahl, C., & Jamshidi, P. (2016). *Microservices: A Systematic Mapping Study*. In CLOSER (pp. 137-146).